# Flappy Universe: A VHDL Game Console on the DE0-CV FPGA

*Team T13 – Team Resistors, COMPSYS 305 Digital Systems Design*

Dohyun Kim
Dept. of Electrical, Computer and Software Engineering
The University of Auckland
Auckland, New Zealand
*dkim766@aucklanduni.ac.nz*

Junha Park
Dept. of Electrical, Computer and Software Engineering
The University of Auckland
Auckland, New Zealand
*jpar483@aucklanduni.ac.nz*

Brian Wang
Dept. of Electrical, Computer and Software Engineering
The University of Auckland
Auckland, New Zealand
*bwan169@aucklanduni.ac.nz*

**Abstract: This report presents Flappy Universe, a single-player side-scrolling game implemented entirely in digital hardware on a Terasic DE0-CV board using VHDL. The game reproduces the core mechanic of Flappy Bird: the player taps a PS/2 mouse button to move a chosen character upward. At the same time, gravity pulls it down, steering through a stream of randomly generated pipe gaps shown on a 640×480 VGA display. The design is organised as a control unit and a datapath. A finite state machine (FSM) sequences the game through title, character-select, play, pause, and game-over states. At the same time, the datapath handles sprite physics, pipe scrolling, axis-aligned collision detection, scoring, and a pseudo-random number generator based on a linear-feedback shift register (LFSR). The game offers a no-damage Training mode and three progressively harder modes (Easy, Medium, and Hard) that introduce a lives system, periodic gravity inversion, and randomly placed plant hazards. Graphics are produced from MIF-initialised ROM sprites, layered over a multi-speed parallax background, via a fixed-priority colour multiplexer. The complete console runs from a single 25 MHz pixel clock generated by an on-chip PLL.**

**Keywords:** FPGA; VHDL; VGA; PS/2 mouse; finite state machine; datapath; LFSR; digital systems design; Terasic DE0-CV; PLL; ROM; MIF

## I. Introduction

The goal of this project was to design a simple game console and a built-in computer game using only digital logic, with no embedded processor or software running on the board. Our chosen game, Flappy Universe, is a side-scroller in which a player keeps a character airborne by tapping a mouse button while avoiding a continuous stream of obstacles. The game is displayed on VGA at the standard 640×480 resolution and controlled with a PS/2 mouse, along with the on-board pushbuttons and seven-segment displays of the DE0-CV.

Designing the game purely in hardware forced a clear separation between the control unit, which decides what the system should be doing at any moment, and the datapath, which performs arithmetic and comparisons to move sprites, detect collisions, and keep score. This report follows that structure. We first describe the game and its requirements, then the overall system architecture and clocking, the controlling finite state machine, and the individual datapath blocks. We close with the main design decisions and trade-offs, an account of resource usage and timing, and suggestions for future work.

## II. Game Description and Requirements

Flappy Universe keeps the familiar one-button mechanic of the original game. The character occupies a fixed horizontal position near the centre of the screen, and the world scrolls past it from right to left. Each press of the left mouse button gives the character an upward impulse; releasing it lets a constant downward acceleration take over, so the player must tap repeatedly to stay aloft. Pipes appear in pairs with a vertical gap, and the player scores one point for every pair successfully passed.

### *A. Game modes and levels*

On power-up, the console shows a title screen (Fig. 3) that doubles as a mode selector. Rather than a DIP switch, we let the player click one of four glowing portals: Training, Easy, Medium or Hard with the mouse. The handout permits sensible changes to the rules, and a clickable menu both reads more naturally and demonstrates live use of the mouse coordinates. Training is a practice mode at the gentlest setting, with the damage system disabled so that the player can rehearse the controls indefinitely. The three game modes form the required minimum of three levels and differ by the hazards they enable: Easy uses pipes and a life health system that is determined by DIP switches; Medium adds periodic gravity inversion; and Hard adds both gravity inversion and randomly spawned plant hazards on the pipe mouths.

### *B. Lives, hazards and gifts*

In the three game modes, the character starts with a set number of lives determined by DIP switches. Touching a pipe body or a plant costs one life and triggers a short flashing invincibility window, so a single contact cannot drain several lives at once. Flying into the top or bottom of the play area counts as an instant loss of all lives, mirroring the original rule that hitting the ground is fatal. A collectable "fries" gift periodically spawns inside the safe gap of the nearest pipe and scrolls along with the world; catching it restores one life up to the maximum, giving the player a reason to take small risks. When the last life is lost, the FSM moves to a game-over screen; in Training, the life count never reaches zero, so play continues until the player resets.

*C. Controls and on-screen information*

The mouse's left button is the single gameplay control. The pushbuttons provide console functions: KEY0 acts as a global reset back to the title screen; KEY2 pauses, and KEY3 resumes play. On-screen text rendered from a character ROM shows the current mode and a live "SCORE" readout, and the three seven-segment displays mirror the score as a decimal value, so the board itself reports the game state even without looking at the monitor.

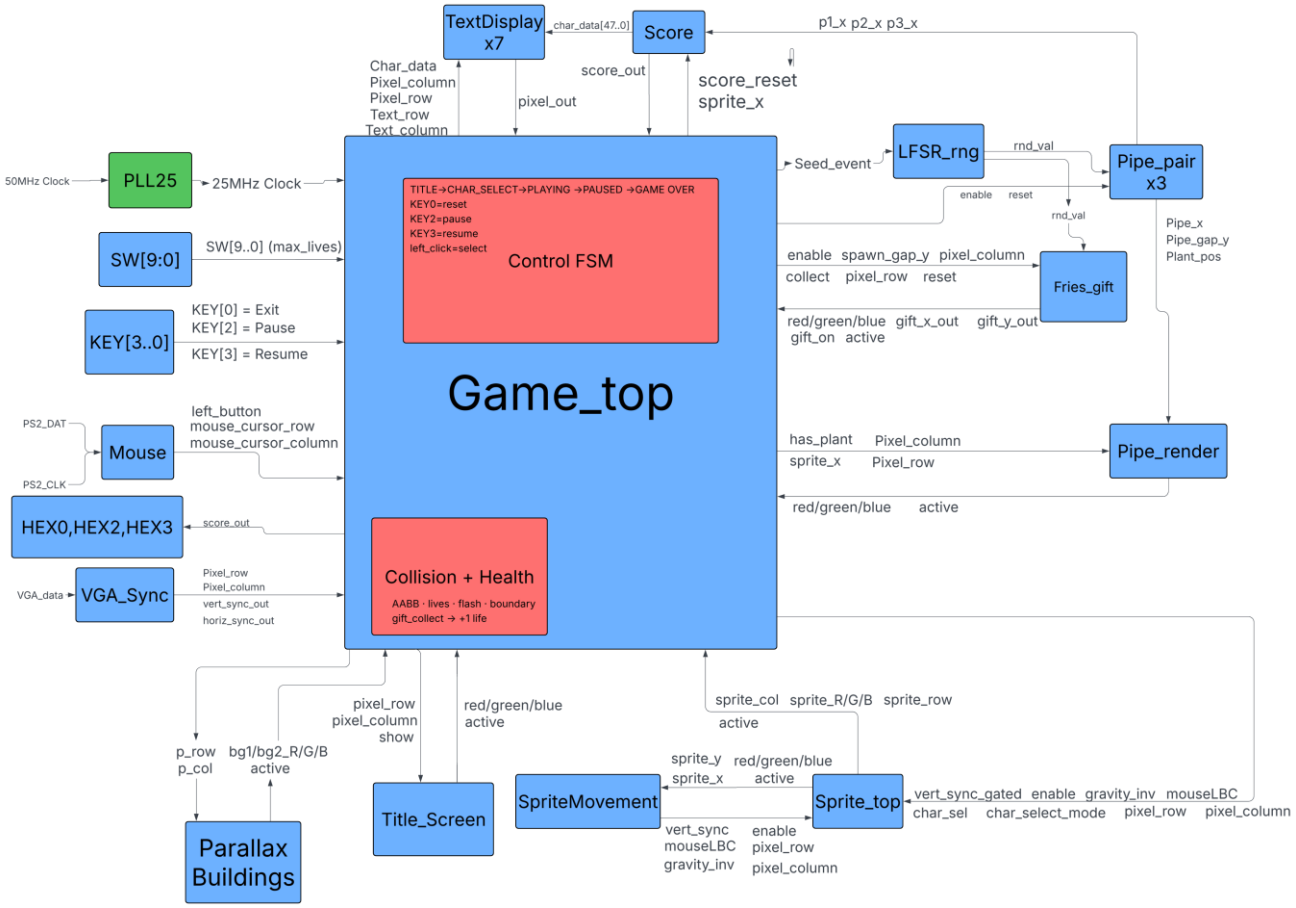



Fig. 1. Top-level block diagram of the Flappy Universe console, separating the control unit (FSM) from the datapath, graphics pipeline and the provided clock, mouse and VGA interface blocks.

## III. System Architecture

The top-level entity game_top integrates every component and connects them to the physical board pins. Fig. 1 shows the overall organisation, with the control unit on the left, the datapath in the centre, and the graphics pipeline feeding the VGA interface on the right.

A. Clocking and the synchronous base

The board's 50 MHz oscillator is halved to a single 25 MHz pixel clock by an on-chip phase-locked loop (PLL25), generated as a Quartus IP block. This 25 MHz clock essentially drives the entire design, so the system is single-clock-domain, and we avoid the metastability and synchronisation issues that come with crossing clock domains. The 25 MHz rate is exactly the dot clock the 640×480 VGA mode expects, so no further division is needed for the display. A second, slower timebase is derived from the vertical-sync pulse: character physics and several timers advance once per frame at the 60 Hz refresh rate, decoupling the speed of motion from the pixel clock and making the game feel consistent.

*B. Provided interface blocks*

Three blocks supplied with the project were reused unchanged and are clearly delimited from our own work. VGA_SYNC produces the horizontal and vertical sync pulses, and the running pixel row and column counters that index every pixel of the 640×480 frame. The PS/2 mouse interface decodes the serial mouse packets into a debounced left-button signal and a pair of cursor coordinates, which the design reuses both as the flap input during play and as the pointer that drives the menu screens. The character ROM holds an 8×8 font that our text block indexes. Everything else, including the FSM, sprite physics, the pipe generator, collision and scoring, the LFSR, the parallax background, the gift logic and the priority multiplexer, was written for this project.

*C. Display layering*

Every on-screen pixel is determined by a single fixed priority colour multiplexer evaluated at the current pixel position. Damage and gravity-inversion border flashes sit at the top of the priority order, followed by the heart life indicator, on-screen text and the mouse cursor, then the active character sprite, the gift, the title artwork, the pipes,

and finally the two parallax background layers at the bottom. Each graphics block must report only its colour and whether it is opaque at the current pixel; the multiplexer resolves all overlaps in a single place. Centralising the compositing this way made the visual behaviour easy to reason about and to extend as features were added.

## IV. Control Unit: The Game FSM

The control unit is a synchronous finite-state machine clocked at 25 MHz. It owns the flags (show title, char_select_mode, game running, game paused, game over and the difficulty code) that the rest of the design reads as enables and mode selectors, so there is exactly one place that decides which phase of the game is active. Fig. 2 summarises its states and transitions.

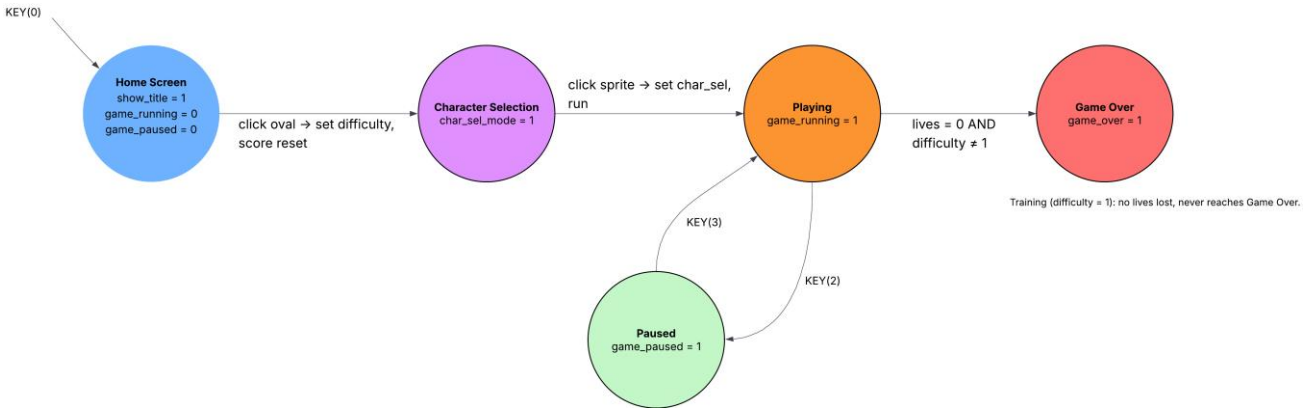



Fig. 2. High-level finite state machine controlling game flow. KEY0 forces a return to TITLE from any state; KEY2/KEY3 toggle the PAUSED sub-state during play.

From the TITLE state, clicking a portal records the chosen difficulty and advances to CHARACTER SELECT. There, the player clicks one of three characters (a frog, the classic Flappy Bird, or a chat-bubble icon), which locks the selection and enters PLAYING. During play, KEY2 enters a PAUSED sub-state that freezes scrolling and physics and blanks off the vertical sync, so motion stops cleanly, and KEY3 returns to PLAYING. Losing the last life (in any non-Training mode) moves the machine to GAME OVER, which displays a losing screen until KEY0 is pressed. KEY0 is a global reset: from any state, it returns the console to TITLE and clears the score, lives and difficulty. Menu transitions are guarded by hit-box comparisons on the latched mouse coordinates, so only a click inside a portal or a character causes a transition.

## V. Datapath Components

*A. Character physics*

Each character is updated once per video frame on the rising edge of the vertical-sync signal, tying the motion to the 60 Hz refresh rate and keeping it smooth and frame-rate independent of the pixel clock. A signed vertical velocity variable is integrated to obtain the vertical position at each frame. A button press sets a fixed upward velocity of about 9 pixels per frame; when the button is released, an acceleration of 3 pixels per frame is added, capped at a terminal speed of 8 pixels per frame, producing the characteristic falling arc of the original game. The position is clamped to the playfield (roughly rows 0 to 470) and is reset to the vertical centre on the rising edge of the game-running enable. A separate inverted-gravity branch flips the sign of the impulse and acceleration so that, during inversion, tapping pushes the character down while it drifts upward when idle.

*B. Scrolling pipes and randomness*

Three instances of a pipe_pair block create obstacles, initialised at horizontal positions of 640, 860, and 1080 pixels (about 220 pixels apart), forming a steady, evenly spaced stream. Each instance scrolls left by one pixel every 100,000 clock ticks, or about 250 pixels per second at 25 MHz. When a pipe leaves the left edge, it recycles to the right edge and samples the random source to choose a fresh gap centre within an allowed band (about rows 80-320). The random source is a 16-bit Fibonacci LFSR implementing the maximal-length polynomial $x^{16} + x^{15} + x^{13} + x^4 + 1$, which cycles through all 65,535 non-zero states. To make every playthrough different, a free-running counter is captured into the LFSR on the player's first click, so the sequence depends on human reaction time rather than always starting from the same seed.

*C. Collision detection and scoring*

Collision detection is combinational and uses axis-aligned bounding-box tests. For each pipe, the logic checks whether the character's fixed horizontal band overlaps the pipe and, if so, whether the character lies inside the safe vertical gap; an overlap outside the gap is a hit. Plant hazards in Hard mode are tested with a second bounding box anchored to the top or bottom of the gap. A separate boundary-check flag indicates contact with the top or bottom of the screen. A clocked health process edge-detects these hit signals so that one contact removes exactly one life, then asserts a short invincibility window (about 60 ms) and a flash (about 30 ms) so a single touch cannot drain several lives. The same process adds a life when the gift is collected, and a contact with the screen edge zeroes the life count outright. Making this process the single writer of the lives signal avoided the multiple-driver errors that otherwise arise when several events change the same register. The score block counts a point each time a pipe's trailing edge passes the character, saturates at 999, and formats the value into three decimal digits for both the on-screen "SCORE" text and the seven-segment displays.

*D. Difficulty scaling and gravity inversion*

Difficulty is expressed through the hazards each mode enables rather than through raw speed. Training turns off the entire damage path, so the player cannot lose. Easy adds to the life health system with DIP switch management. Medium and Hard add a periodic gravity inversion: a free-running timer waits a randomised interval (roughly three to eight seconds, derived from the LFSR), shows a three-second "3-2-1" countdown drawn with the text block, then inverts gravity for about four seconds while a coloured border warns the player, before returning to normal. Hard additionally gives each recycled pipe a 50% chance of carrying a plant at the top or bottom of its gap, determined by individual LFSR bits, which sharply narrows the safe path.

*E. Graphics generation*

Sprites for the three selectable characters (a frog, the classic flappy bird, and a chat-bubble icon) and the title artwork are stored as ROMs initialised from MIF files generated from PNG art using a small Python converter. The background is built from two independently scrolling layers: distant buildings and nearer buildings, moving at different rates to create a parallax sense of depth. Text is drawn by indexing the provided character ROM through a reusable TextDisplay block whose character count and pixel scale are generic parameters, so the same block renders the small score readout, the larger mode labels, the inversion countdown and the pause and game-over banners.




Fig. 3. Title and mode-select screen. The four portals are mouse-clickable hitboxes that select Training, Easy, Medium or Hard.

## VI. Design Decisions and Trade-offs

Several choices shaped the final design. We fixed the character's horizontal position and scrolled the world instead, which is the standard simplification for this style of game and keeps the rendering and collision maths in one dimension of motion. We chose a mouse-driven, clickable menu over a DIP-switch mode for selecting because it gave a more polished feel and exercised the mouse coordinate path that the gameplay also relies on. We differentiated the levels mainly through mechanics (enabling damage, then gravity inversion, then plant hazards) rather than only through speed; in the current build, the scroll rate is held constant at roughly 250 pixels per second across modes, and varying it per level is the most natural next refinement. Tying physics to the vertical sync rather than the pixel clock kept motion smooth and predictable, and seeding the LFSR from the first click was a cheap way to guarantee variety between games. Concentrating all colour decisions in a single priority multiplexer and all life changes in a single process traded a little extra wiring for much simpler, more debuggable behaviour.

### VII. Implementation and Verification

The game was built incrementally rather than all at once. The interim milestone was used to confirm every input and output interface in isolation before any gameplay was added: the decoded mouse packet was shown moving a cursor, and on the seven-segment displays, the pushbuttons and DIP switches were exercised, a controllable ball was drawn over VGA to prove the display path, and text of different sizes was rendered through the character ROM. The PLL and timing blocks were checked using the vendor simulation flow. At the same time, the integrated game logic, including physics feel, collision boundaries, scoring, life and invincibility behaviour, and each FSM transition, was verified directly on the DE0-CV through playtesting each mode. Building and testing one block at a time made faults easy to localise: because all colour decisions pass through a single multiplexer and all life changes through a single process, most visual or scoring bugs could be traced to one place.

## VIII. Resource Utilisation and Performance

The design targets the DE0-CV's Cyclone V 5CEBA4F23C7 device. It is comfortably small in logic terms, occupying 2,198 of the 18,480 adaptive logic modules (12%) and 892 registers, and using just 2 of the 66 DSP blocks (3%), inferred from the small amount of divide-and-modulo arithmetic in the score-digit extraction and the timing logic. The clear pressure point is embedded memory: the design consumes 2,238,464 of the 3,153,920 available block-memory bits (71%), almost all of it spent on ROMs initialised from MIF files. The full-screen title artwork is by far the largest single consumer, followed by the per-character sprite ROMs and the background layers, which is why memory dominates the footprint while logic stays low. A single PLL (1 of 4) generates the clock, and 53 of 224 I/O pins are used. Timing is undemanding: the entire design runs from a single 25 MHz clock, giving a 40 ns period and ample slack against the longest combinational paths, which lie in the collision and score arithmetic, so the maximum frequency reported by the Timing Analyser sits well above the 25 MHz the design actually needs. Table I summarises these figures.

TABLE I. Resource Utilisation and Timing Summary

| Metric | Value |
|---|---|
| Logic (ALMs) | 2,198 / 18,480 (12%) |
| Total registers | 892 |
| Block memory bits | 2,238,464 / 3,153,920 (71%) |
| DSP blocks | 2 / 66 (3%) |
| I/O pins | 53 / 224 (24%) |
| PLLs used | 1 / 4 (25%) |
| Pixel clock | 25 MHz |
| Restricted Fmax | 46.65 MHz |

*Resource figures from the Quartus Prime Fitter summary*

# IX. CONCLUSION AND FUTURE WORK

Flappy Universe meets the project requirements: a fully hardware-described game console with a clear control-unit/datapath split, two operating modes (a Training mode and a game mode with three difficulty levels), mouse and pushbutton control, VGA graphics with text and animated sprites, a parallax background, an LFSR-based random source, and on-screen and seven-segment status reporting. The FSM cleanly sequences the experience from menu to game-over, and the fixed-priority renderer made it straightforward to add features incrementally.

The most worthwhile future improvement is to scale the scroll speed with the level so that difficulty ramps through pace as well as hazards. Beyond that, compressing or down sampling the title-screen ROM would free embedded memory; adding sound and a persistent high score would improve polish; and a wider variety of obstacles and gifts would extend replay value.

## References


[1] Terasic Technologies, “DE0-CV Development and Education Board User Manual,” 2015.

[2] Intel Corporation, “Cyclone V Device Handbook, Volume 1: Device Interfaces and Integration,” 2018.

[3] M. Hemmati and M. Biglari-Abhari, “COMPSYS 305 Digital Systems Design – Mini-Project Handout,” Dept. of Electrical, Computer and Software Engineering, The University of Auckland, 2026.

[4] dotGears, “Flappy Bird,” mobile game, 2013.

[5] P. Alfke, “Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators,” Xilinx Application Note XAPP052, 1996.

[6] 225 Serial Character LED Matrix :: Quicker, easier and cheaper to make your own chip!. https://www.tinytapeout.com/runs/tt07/tt_um_ccattuto_char matrix